

Introduction to R

Shu Guo

Thursday, January 07, 2016

YAML

Introduction to R

R is a language and environment for statistical computing and graphics. It is a GNU project which is similar to the S language and environment which was developed at Bell Laboratories (formerly AT&T, now Lucent Technologies) by John Chambers and colleagues. R can be considered as an open source implementation of S.

R provides a wide variety of statistical and graphical techniques, and is highly extensible. One of R's strengths is the ease with which well-designed publication-quality plots can be produced, including mathematical symbols and formulae where needed.

R is available as Free Software under the terms of the Free Software Foundation's GNU General Public License in source code form. It compiles and runs on a wide variety of UNIX platforms and similar systems (including FreeBSD and Linux), Windows and MacOS.

Installing R and RStudio

In this section we will learn how to install R and RStudio, and some basic knowledge of R programming.

Installing Base R

We can go to the R Foundation website, <https://www.r-project.org/>, and choose one of the mirrors that close to you to download R.

Since I am close to Toronto so I will choose University of Toronto and it will be faster to download. If you installed a LINUX distribution, most like the R will come with the system when you installing it, and you can select base installer for Mac or Windows.

After installed R, the first time you opened R you will see a menu bar, a tool and R console. On the console you can see R version, license and other related information. The `>` sign on the bottom means that R is ready to accept commands here. You can type your code here or open a file to write/execute your code and save the code later.

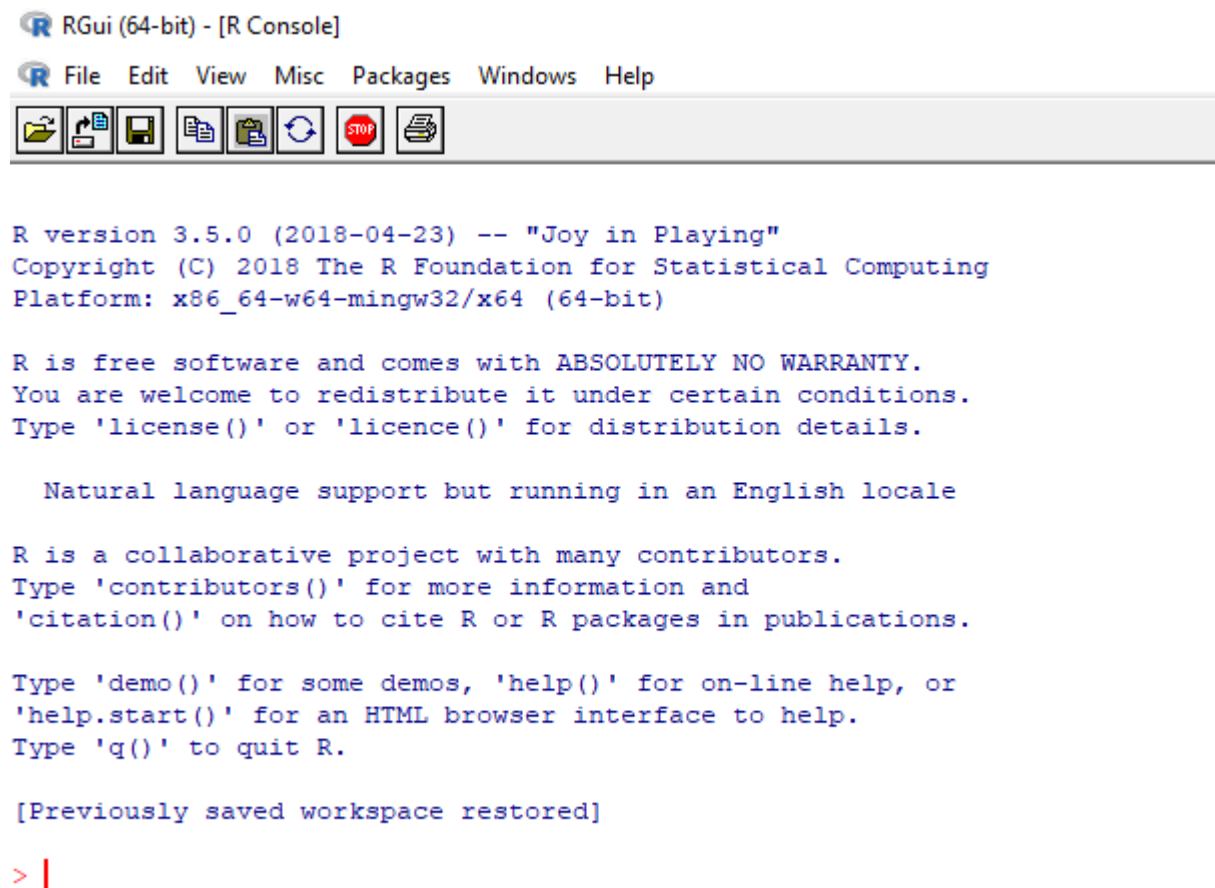


Figure 1: R Console

Installing RStudio

This is the R GUI with limited functionality, and I would like to recommend a R IDE called RStudio. You can go to the following webpage to download RStudio: <https://www.rstudio.com/products/RStudio/>, and then you can just install RStudio like any other software.

The first time you open RStudio will be like this:

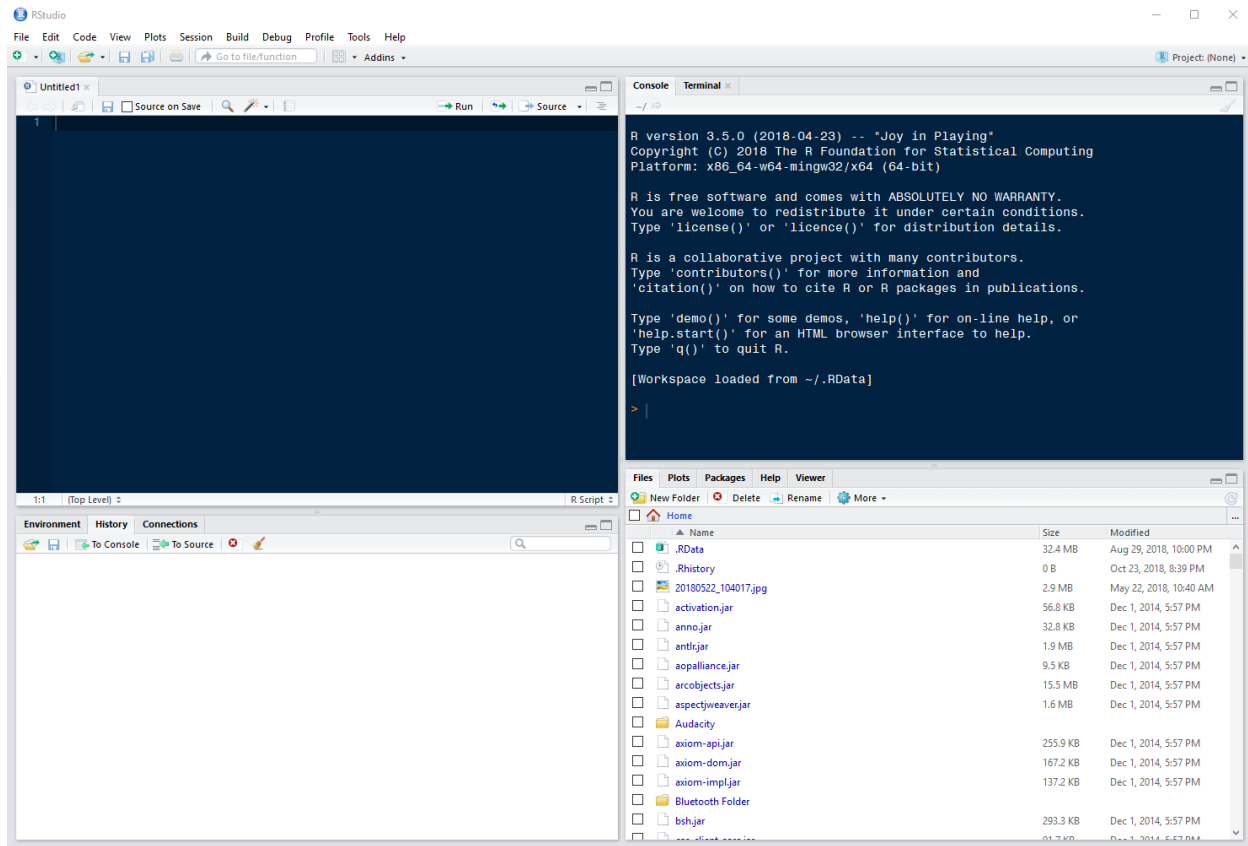


Figure 2: RStudio

We can see there are four sections on the screen, the first section on the top left is the place that you can write your code, and the result will display on the console to the top right section. Other sections can have information for environment, files, packages, history, etc. We can change the layout in Global Options -> pane layout, and we can also change Appearance and Version Control here.

R CRAN (The Comprehensive R Archive Network) has more 10,000 R packages. use “install.package()” to install new packages.

R Basics

```
# We can use R as a calculator
(10 + 5 - 3)*2
```

```
## [1] 24
```

```
# Assigning scalar variables using <-  
a <- 5*3  
print(a)
```

```
## [1] 15
```

```
# Create vectors using c()  
b <- c(3, 1, 6)  
print(b)
```

```
## [1] 3 1 6
```

```
c <- c("Monday", "Tuesday", "Wednesday")  
print(c)
```

```
## [1] "Monday"      "Tuesday"      "Wednesday"
```

```
# Special numbers: 1. NaN, not-a-number 2. NA, not available  
sqrt(-2)
```

```
## Warning in sqrt(-2): NaNs produced
```

```
## [1] NaN
```

```
b <- NA  
print(b)
```

```
## [1] NA
```

```
# finding help using ?, ?? or help(), help.search()  
# ?mean  
# ??plot
```

Data Structure

Vector

Vector is the basic data structure in R. There are two types of vectors in R: atomic vector and list or generic vector. They are all one dimensional data structure and atomic vector is homogenous while list is heterogeneous.

Atomic Vector

There are four common atomic vectors in R: logical, integer, double(numeric), character, complex and raw, and we will discuss the first four types here. All elements in the same atomic vector must be in the same type. If different types of values were put in the same atomic vector, then the values will be coerced to the most flexible type. The rank of types from most to the least flexible is: character, double, integer, logical.

```
# create a logical vector
vct_loc <- c(TRUE, T, F, FALSE, T)
print(sum(vct_loc))
## [1] 3

# create a integer vector
vct_int <- c(3L, 2L, 9L, 5L)
print(vct_int + 1)
## [1] 4 3 10 6

# create a double vector
vct_dbl <- c(32, 6.7, 8.3, 10.1)
print(vct_dbl * 2)
## [1] 64.0 13.4 16.6 20.2

# create a character vector
vct_chr <- c("today", "tomorrow", "yesterday")
print(which(vct_chr == "yesterday"))
## [1] 3

# determine type using typeof()
typeof(vct_int)
## [1] "integer"

# check if it is a specific type use: is.character(), is.double(),
# is.integer(), is.logical()
is.double(vct_dbl)
## [1] TRUE

# or use is.atomic()
is.atomic(vct_loc)
## [1] TRUE

# find number of elements
length(vct_loc)
## [1] 5

# Coerced to the most flexible type: character
vct_cred <- c(TRUE, 3.8, "New")
print(vct_cred)
```

```
## [1] "TRUE" "3.8" "New"
typeof(vct_cred)
## [1] "character"
```

Exercise R actually no scalar type. Try to create a single value object such as `a <- 1` and test if it is a vector.

Attributes

All objects can have additional attributes such as `name`, `class`, `comment`, `dim`, `dimnames`, `names` etc. One of the important attributes is the factor levels that defined factors. A factor is a vector that can only contain predefined categorical values.

```
# set names attribute
loc_na <- c("a", "b", "c", "d", "e")
names(vct_loc) <- loc_na

# add comments
comment(vct_loc) <- "This is a logical vector"

print(vct_loc)
```

```
##      a      b      c      d      e
## TRUE  TRUE FALSE FALSE  TRUE
```

```
attributes(vct_loc)
```

```
## $names
## [1] "a" "b" "c" "d" "e"
##
## $comment
## [1] "This is a logical vector"
```

```
# create a factor vector
fct <- c("low", "high", "medium", "high", "low", "medium",
        "high", "low")
fct_vct <- factor(fct, order = TRUE,
                 levels = c("low", "medium", "high"))
levels(fct_vct)
```

```
## [1] "low"      "medium" "high"
```

```
as.numeric(fct_vct)
```

```
## [1] 1 3 2 3 1 2 3 1
```

Excercise: Why is `73 == "73"` is ture? Why `2 > TRUE` true?

Matrix and Array

Array and matrix are also homogenous which means each elements in an array or matrix must haev the same mode(logic, numeric or logic). Array is multi-dimensional which matrix is two dimentional

```
# Create a Three dimentional array
m1 <- c("A1", "A2")
m2 <- c("B1", "B2", "B3")
m3 <- c("C1", "C2", "C3", "C4")
arr <- array(1:24, c(2, 3, 4), dimnames = list(m1, m2, m3))
print(arr)
```

```
## , , C1
##
##      B1 B2 B3
## A1   1  3  5
## A2   2  4  6
##
## , , C2
##
##      B1 B2 B3
## A1   7  9 11
## A2   8 10 12
##
## , , C3
##
##      B1 B2 B3
## A1  13 15 17
## A2  14 16 18
##
## , , C4
##
##      B1 B2 B3
## A1  19 21 23
## A2  20 22 24
```

```
# Matrices
max1 <- matrix(1:6, 3, 2)
print(max1)
```

```
##      [,1] [,2]
## [1,]    1    4
## [2,]    2    5
## [3,]    3    6
```

```
max2 <- 1:8
dim(max2) <- c(2, 4)
print(max2)
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    3    5    7
## [2,]    2    4    6    8
```

```
# Matrix computation using %*%
print(max1 %*% max2)
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    9   19   29   39
## [2,]   12   26   40   54
## [3,]   15   33   51   69
```

Data Frame

Data frame is the most common way to store the data for analysis in R, with each row is a record and each column is variable that can only have one type of data.

```
df <- data.frame(x = 1:5, y = letters[1:5],
                 z = runif(1) > 0.5,
                 row.names = c("row1", "row2", "row3", "row4", "row5"),
                 stringsAsFactors = FALSE)
print(df)
```

```
##      x y      z
## row1 1 a TRUE
## row2 2 b TRUE
## row3 3 c TRUE
## row4 4 d TRUE
## row5 5 e TRUE
```

```
# check if an object is a data frame using class or is.data.frame.
class(df)
```

```
## [1] "data.frame"
```

```
is.data.frame(df)
```

```
## [1] TRUE
```

List

Lists are one dimensional heterogeneous vector that can have any kind of objects. For example, a list may contain a combination of vectors, matrix, data frames and other lists.

```
# Create a list
a <- "A New List"
b <- c(1, 1, 2, 5, 14, 42)
c <- matrix(1:10, nrow = 5)
d <- list("one", "two", "three")
list1 <- list(title = a, b, c, d, month.name)
names(list1)[2] <- "Second item"
print(list1)
```

```
## $title
## [1] "A New List"
##
## $'Second item'
## [1] 1 1 2 5 14 42
##
## [[3]]
##      [,1] [,2]
## [1,]    1    6
## [2,]    2    7
## [3,]    3    8
## [4,]    4    9
## [5,]    5   10
##
## [[4]]
## [[4]][[1]]
## [1] "one"
##
## [[4]][[2]]
## [1] "two"
##
## [[4]][[3]]
## [1] "three"
##
##
## [[5]]
## [1] "January" "February" "March"    "April"    "May"      "June"
## [7] "July"     "August"   "September" "October"  "November" "December"
```

```
#lists are used to build up other data structures such as data frames and statistical model ob.  
is.list(df)
```

```
## [1] TRUE
```

```
# Use unlist to convert a list to atomic vector.  
is.atomic(unlist(df))
```

```
## [1] TRUE
```

Importing and Exporting Data

```
# Read data from text file  
dat1 <- read.table("data/dat1.txt", header = TRUE, stringsAsFactors = FALSE)  
str(dat1)
```

```
## 'data.frame':    6 obs. of  3 variables:  
## $ age      : int  0 1 2 0 1 2  
## $ weight: num  1.2 2.3 3.5 0.9 1.9 2.8  
## $ gender: chr  "male" "male" "male" "female" ...
```

```
# Read data from csv file  
hourly <- read.csv("data/hourly.csv", skip = 15,  
                  stringsAsFactors = FALSE, header = TRUE)
```

```
# Read data from excel file using read.xlsx in xlsx package  
library(xlsx)  
dat2 <- read.xlsx("data/infor.xlsx", sheetIndex = 1)  
print(dat2)
```

```
##      ID Fname Lname      Phone      Email  
## 1 A001 Nancy  Chen 4162279388  sinolink@yesic.com  
## 2 A002 Yubo  Zheng 9054718502  zhengybo@yahoo.com  
## 3 A003 Cathy  Zhang 4163353499  cathyzq_2000@yahoo.com  
## 4 A004 Bin    Hu   4162922570  hubinsas@hotmail.com
```

```
# Import data from excel file using readxl package  
library(readxl)  
dat2 <- read_xlsx("data/infor.xlsx", sheet = 1)  
print(dat2)
```

```
## # A tibble: 4 x 5
##   ID      Fname Lname      Phone Email
##   <chr> <chr> <chr>      <dbl> <chr>
## 1 A001  Nancy Chen  4162279388 sinolink@yesic.com
## 2 A002  Yubo  Zheng  9054718502 zhengybo@yahoo.com
## 3 A003  Cathy Zhang  4163353499 cathyzq_2000@yahoo.com
## 4 A004  Bin    Hu    4162922570 hubinsas@hotmail.com
```

```
# Import data from excel file, change the column names, skip the first row(drop
# original column names), change missing to NA, and set the column types
read_excel(
  "data/students.xlsx",
  col_names = c("student_id", "full_name", "favourite_food", "meal_plan", "age"),
  skip = 1,
  na = c("", "N/A"),
  col_types = c("numeric", "text", "text", "text", "text")
)
```

```
## # A tibble: 6 x 5
##   student_id full_name      favourite_food meal_plan      age
##         <dbl> <chr>          <chr>          <chr>      <chr>
## 1           1 Sunil Huffmann Strawberry yoghurt Lunch only    4.0
## 2           2 Barclay Lynn   French fries    Lunch only    5.0
## 3           3 Jayendra Lyne   <NA>           Breakfast and lunch 7.0
## 4           4 Leon Rossini    Anchovies       Lunch only    <NA>
## 5           5 Chidiegwu Dunkel Pizza           Breakfast and lunch five
## 6           6 Güvenç Attila    Ice cream       Lunch only    6.0
```

```
# Exporting data using write.table
write.table(dat1, "data/mydata.txt", sep="\t")
```